



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

Ψηφιακά Συστήματα VLSI

3η Εργαστηριακή Άσκηση

Υλοποίηση FIR Φίλτρου

Ομάδα 6

Μέλος 1:

Ονοματεπώνυμο: Γιώργος Μπάρκας

Αριθμός Μητρώου: el22139

Μέλος 2:

Ονοματεπώνυμο: Δημήτρης Βλασακάκης

Αριθμός Μητρώου: el22818

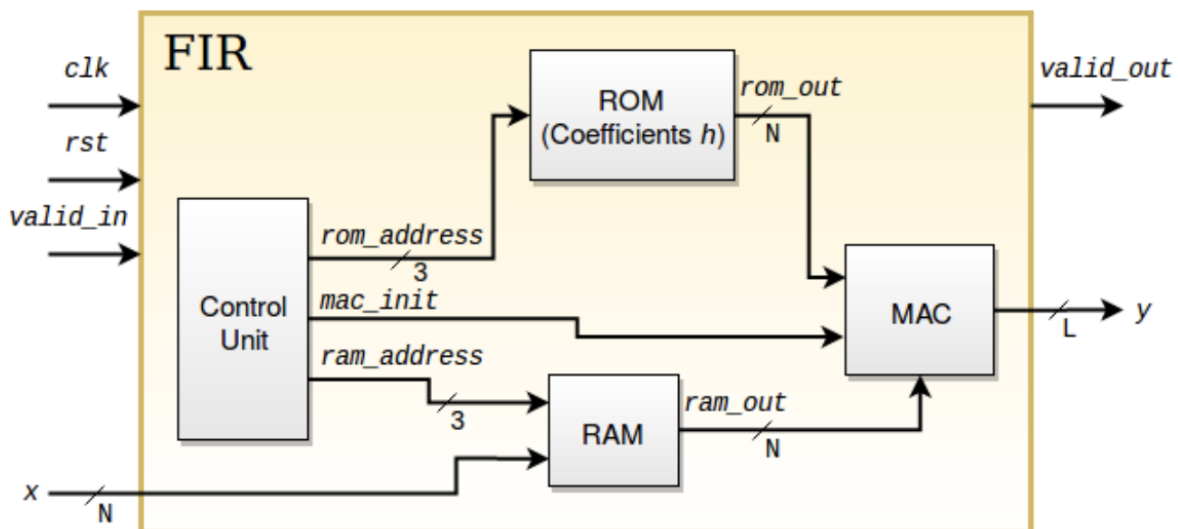
Ένα fir φίλτρο έχει την εξής γενική μορφή:

$$y[n] = \sum_{k=0}^M h[k]x[n-k] = h[0]x[n] + h[1]x[n-1] + \dots + h[M]x[n-M]$$

όπου:

- M : τάξη του φίλτρου
- $y[n]$: η διακριτή έξοδος του φίλτρου την χρονική στιγμή n
- $h[k]$: ο k -οστός συντελεστής του φίλτρου
- $x[n]$: η τιμή του σήματος εισόδου την χρονική στιγμή n

Στόχος μας σε αυτήν την εργαστηριακή άσκηση είναι να υλοποιήσουμε ένα **8-tap fir φίλτρο**. Το τελικό μας κύκλωμα θα έχει την εξής μορφή:



MAC: Μονάδα Πολλαπλασιασμού με Συσσωρευτή (Multiplier Accumulator Unit)

Ο MAC πρακτικά αποτελεί τη μονάδα η οποία παράγει το αποτέλεσμα μας, εκτελώντας εσωτερικά την πράξη:

$$a \leftarrow a + b * x$$

Πρακτικά δέχεται ως είσοδο τους σταθερούς συντελεστές h του φίλτρου, τις τιμές του σήματος εισόδου και ένα *init* σήμα ώστε να ξεκινήσει τη συσσώρευση.

Επίσης, κάθε τιμή εξόδου πρέπει να αποτελείται από $2N + \log N$ bits, αφού η έξοδος εξαρτάται από πολλαπλασιασμούς τιμών των N bits και N προσθέσεις αριθμών των $2N$ bits.

Η υλοποίηση μας φαίνεται παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity mac is
    port(
        clk : in std_logic;
        rom_out: in std_logic_vector(7 downto 0):= (others => '0');
        ram_out: in std_logic_vector(7 downto 0):= (others => '0');
        mac_init : in std_logic;
        mac_out : out std_logic_vector(18 downto 0):= (others => '0')
    );
end mac;

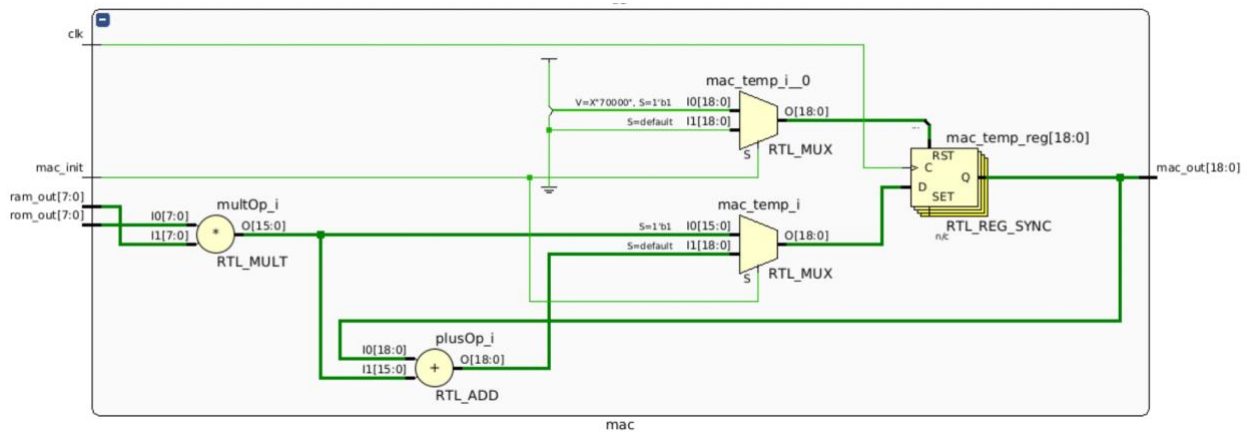
architecture mac_arch of mac is
    signal mac_temp : std_logic_vector(18 downto 0):= (others => '0');
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if mac_init = '1' then

                mac_temp <= EXT(rom_out * ram_out, mac_temp'length);
            else

                mac_temp <= mac_temp + EXT(rom_out * ram_out, mac_temp'length);
            end if;
        end if;
    end process;

    mac_out <= mac_temp;
end mac_arch;
```

Με χρήση του Elaborated Design λαμβάνουμε:



ROM

Η ROM έχει αποθηκευμένους τους σταθερούς συντελεστές h του φίλτρου. Δέχεται στην είσοδο της μια διεύθυνση και δίνει ως έξοδο τον αντίστοιχο συντελεστή, όταν αυτή είναι ενεργοποιημένη.

Στο παράδειγμα που θα δούμε παρακάτω, οι συντελεστές είναι οι εξής:

$$h = [1, 2, 3, 4, 5, 6, 7, 8]$$

Η υλοποίηση μας φαίνεται παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity rom is
    generic (
        coeff_width : integer := 8 -- width of coefficients (bits)
    );
    Port (
        clk      : in  STD_LOGIC;
        en       : in  STD_LOGIC; -- operation enable
        rom_addr  : in  STD_LOGIC_VECTOR (2 downto 0); -- memory address
        rom_out   : out STD_LOGIC_VECTOR (coeff_width-1 downto 0) := (others => '0') -- output data
    );
end rom;

architecture rom_arch of rom is

    type rom_type is array (7 downto 0) of STD_LOGIC_VECTOR (coeff_width-1 downto 0);

    -- Initialize ROM with 8 coefficients
    signal h : rom_type := (
        "00001000",
```

```

        "00000111",
        "00000110",
        "00000101",
        "00000100",
        "00000011",
        "00000010",
        "00000001"
    );

    signal rdata : STD_LOGIC_VECTOR(coeff_width-1 downto 0) := (others => '0');

begin

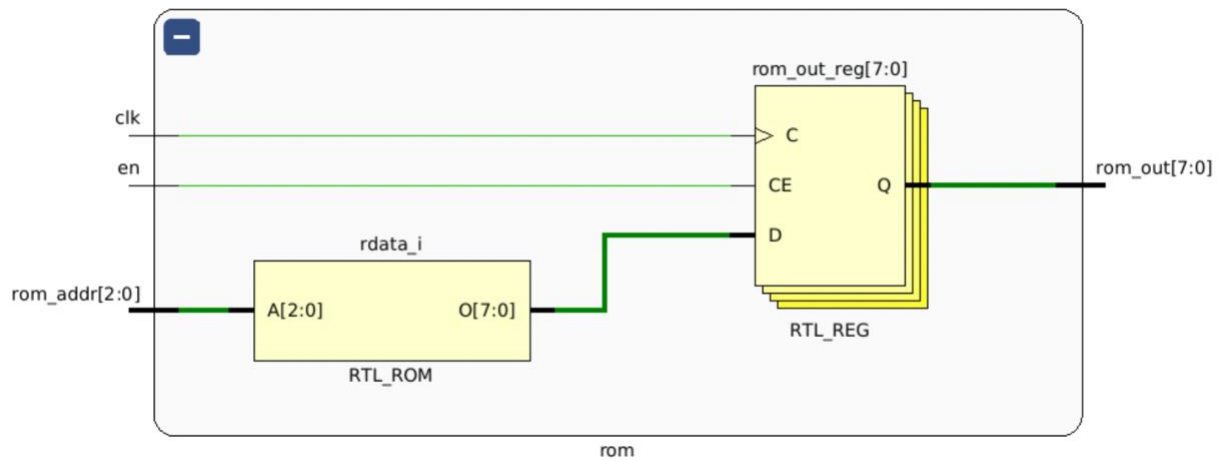
    rdata <= h(to_integer(unsigned(rom_addr)));

    process (clk)
    begin
        if rising_edge(clk) then
            if en = '1' then
                rom_out <= rdata;
            end if;
        end if;
    end process;

end rom_arch;

```

Με χρήση του Elaborated Design λαμβάνουμε:



RAM

Η RAM στη συγκεκριμένη υλοποίηση, λειτουργεί ως μια κυλιόμενη λίστα. Κατά τη λειτουργία εγγραφής, τοποθετεί την πιο πρόσφατη τιμή στην 1^η θέση και ολισθαίνει τις υπόλοιπες κατά μια θέση, με την τελευταία να διαγράφεται ώστε πάντα να κρατάει 8 τιμές.

Κατά τη λειτουργία ανάγνωσης, δέχεται στην είσοδο μια διεύθυνση και δίνει στην έξοδο την αντίστοιχη τιμή.

Η υλοποίηση μας φαίνεται παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ram is
    generic (
        data_width : integer := 8 -- width of data (bits)
    );
    port (
        clk : in  STD_LOGIC;
        we  : in  STD_LOGIC; -- memory write enable
        en  : in  STD_LOGIC; -- operation enable
        ram_address : in  STD_LOGIC_VECTOR(2 downto 0); -- memory address
        x : in  STD_LOGIC_VECTOR(data_width-1 downto 0); -- input data
        ram_out : out STD_LOGIC_VECTOR(data_width-1 downto 0) := (others => '0') -- output data
    );
end ram;

architecture ram_arch of ram is

    type ram_type is array (7 downto 0) of STD_LOGIC_VECTOR(data_width-1 downto 0);
    signal RAM : ram_type := (others => (others => '0'));

begin

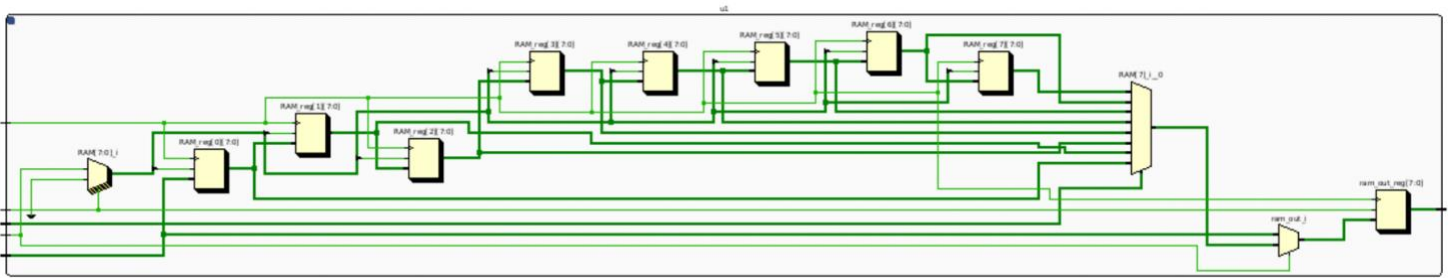
    process (clk)
    begin
        if rising_edge(clk) then
            if en = '1' then
                if we = '1' then -- write operation

                    RAM(0) <= x;
                    for i in 1 to 7 loop
                        RAM(i) <= RAM(i-1);
                    end loop;

                    ram_out <= x;
                else -- read operation
                    ram_out <= RAM(to_integer(unsigned(ram_address)));
                end if;
            end if;
        end if;
    end process;

end ram_arch;
```

Με χρήση του Elaborated Design λαμβάνουμε:



Control Unit - Μονάδα Ελέγχου

Η Μονάδα Ελέγχου είναι υπεύθυνη για την σωστή τροφοδότηση διευθύνσεων στις μονάδες ROM και RAM, όπως και για την αποστολή του σήματος *init* του MAC που αναφέραμε προηγουμένως.

Πρακτικά λειτουργεί ως ένας μετρητής 8 τιμών.

Η υλοποίηση μας φαίνεται παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity control_unit is
    port (
        clk : in std_logic;
        rst : in std_logic := '0';
        valid_in : in std_logic;
        mac_init : out std_logic;
        rom_address : out std_logic_vector(2 downto 0);
        ram_address : out std_logic_vector(2 downto 0);
        mem_en : out std_logic
    );
end control_unit;

architecture control_unit_arch of control_unit is
    signal up_counter : std_logic_vector(2 downto 0);
    signal en : std_logic := '0';

begin
    process(clk, rst, valid_in)
    begin
        if rst = '1' then
            up_counter <= (others => '0');
            mac_init <= '0';
            en <= '0';

        elsif clk'event and clk = '1' then
            if en = '1' then
                up_counter <= up_counter + 1;
                if up_counter = "111" then
                    en <= '0'; -- stop counting after reaching 7
                end if;
            end if;
        end if;
    end process;
end control_unit_arch;
```

```

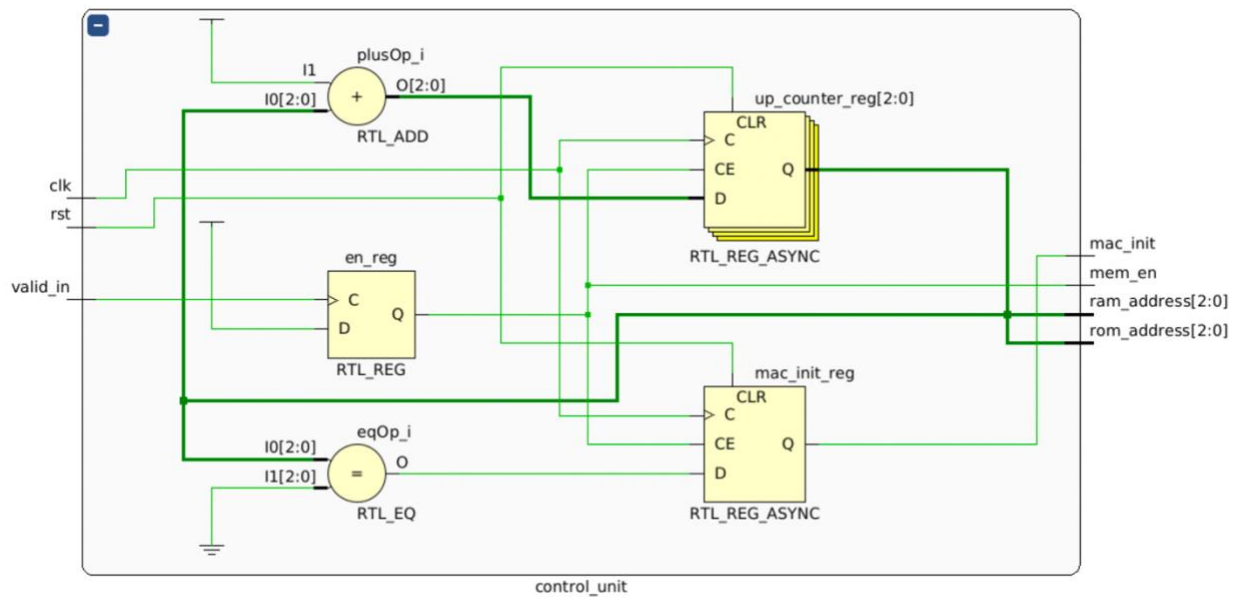
        mem_en <= '0';
    end if;
    if up_counter = "000" then
        mac_init <= '1'; -- initialize MAC at the start of counting
    else
        mac_init <= '0';
    end if;
end if;
end if;
if rising_edge(valid_in) then
    en <= '1'; -- start counting on valid_in signal
    mem_en <= '1';
end if;

end process;
rom_address <= up_counter;
ram_address <= up_counter;

end control_unit_arch;

```

Με χρήση του Elaborated Design λαμβάνουμε:



ΣΧΕΔΙΑΣΗ FIR ΦΙΛΤΡΟΥ

Η λειτουργία του φίλτρου είναι η ακόλουθη:

- Δέχεται μία νέα μη προσημασμένη (unsigned) τιμή εισόδου x κάθε 8 κύκλους ρολογιού.
- Κάθε νέα τιμή της εξόδου y παράγεται με 8 κύκλους ρολογιού διαφορά από την προηγούμενη.
- Για κάθε νέα είσοδο x δέχεται ένα σήμα εγκυρότητας εισόδου (valid_in) το οποίο υποδηλώνει την εγκυρότητα της συγκεκριμένης εισόδου.
- Για κάθε έγκυρη τιμή της εξόδου y ενεργοποιείται ένα σήμα εγκυρότητας εξόδου (valid_out).
- Περιλαμβάνει ασύγχρονο reset το οποίο αρχικοποιεί τη μονάδα Control Unit, τη μνήμη RAM με '0' καθώς και το σήμα εγκυρότητας εξόδου valid_out.

Έχοντας ήδη σχεδιάσει τα components του fir φίλτρου, για την structural περιγραφή του γράφουμε τον παρακάτω VHDL κώδικα:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity fir is
    Port ( clk : in STD_LOGIC;
          rst : in STD_LOGIC;
          valid_in : in STD_LOGIC;
          x : in STD_LOGIC_VECTOR (7 downto 0);
          valid_out : out STD_LOGIC;
          y : out STD_LOGIC_VECTOR (18 downto 0));
end fir;

architecture structural of fir is
    component ram is
        port(
            clk : in STD_LOGIC;
            we : in STD_LOGIC; -- memory write enable
            en : in STD_LOGIC; -- operation enable
            ram_address : in STD_LOGIC_VECTOR(2 downto 0); -- memory address
            x : in STD_LOGIC_VECTOR(8-1 downto 0); -- input data
            ram_out : out STD_LOGIC_VECTOR(8-1 downto 0) -- output data
        );
    end component;

    component rom is
        port
        (
            clk : in STD_LOGIC;
            en : in STD_LOGIC; -- operation enable
            rom_addr : in STD_LOGIC_VECTOR (2 downto 0); -- memory address
            rom_out : out STD_LOGIC_VECTOR (8-1 downto 0) -- output data
        );
    end component;

    component control_unit is
        port
        (
            clk : in std_logic;
            rst : in std_logic;
            valid_in : in std_logic;
```

```

        mac_init : out std_logic;
        rom_address : out std_logic_vector(2 downto 0);
        ram_address : out std_logic_vector(2 downto 0);
        mem_en : out std_logic
    );
end component;

component mac is
    port
    (
        clk : in std_logic;
        rom_out: in std_logic_vector(7 downto 0);
        ram_out: in std_logic_vector(7 downto 0);
        mac_init : in std_logic;
        mac_out : out std_logic_vector(18 downto 0)
    );
end component;

signal ram_address,rom_address: std_logic_vector(2 downto 0);
signal we,mac_init,mem_en:std_logic;
signal ram_out,rom_out:STD_LOGIC_VECTOR(8-1 downto 0):= (others => '0');
signal x_temp : STD_LOGIC_VECTOR (7 downto 0);
signal out_temp :std_logic:= '0';
signal y_temp : STD_LOGIC_VECTOR (18 downto 0):= (others => '0');
signal valid_d : std_logic := '0';

begin
    u0: control_unit port map (clk => clk,
        rst => rst, valid_in => valid_in,rom_address => rom_address, ram_address=> ram_address,mac_init
=> mac_init,mem_en => mem_en);
    u1: ram port map(clk => clk,x =>x,ram_address => ram_address,we=>valid_in,en=>mem_en,ram_out =>
ram_out);
    u2: rom port map(clk=>clk,rom_addr=> rom_address,en=>mem_en, rom_out => rom_out);
    u3: mac port map(clk=>clk,mac_init => mac_init, rom_out => rom_out, ram_out =>ram_out,mac_out =>
y_temp);

fir:process(clk)
begin
    if rising_edge(clk) then
        -- detect last tap
        if rom_address = "111" then
            out_temp <= '1';
        else
            out_temp <= '0';
        end if;
        -- delay valid
        valid_d <= out_temp;
        -- output
        valid_out <= valid_d;
        if valid_d = '1' then
            y <= y_temp;
        end if;
    end if;
end process;
end structural;

```

Στην συνέχεια , θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fir_tb is
end fir_tb;

architecture Behavioral of fir_tb is

    constant N      : integer := 8;
    constant CLK_PER : time    := 40 ns;

    -- 32-sample square wave LUT (unsigned 8-bit)
    -- HIGH = 255 for first 16 samples, LOW = 0 for next 16 samples
    type lut_type is array (0 to 19) of std_logic_vector(7 downto 0);
    --sample list: 208, 231, 32, 233, 161, 24, 71, 140, 245, 247, 40, 248, 245, 124, 204, 36, 107,
    234, 202, 245
    constant SQUARE_LUT : lut_type := (
        "11010000", -- sample 0: 208
        "11100111", -- sample 1: 231
        "00100000", -- sample 2: 32
        "11101001", -- sample 3: 233
        "10100001", -- sample 4: 161
        "00011000", -- sample 5: 24
        "01000111", -- sample 6: 71
        "10001100", -- sample 7: 140
        "11110101", -- sample 8: 245
        "11110111", -- sample 9: 247
        "00101000", -- sample 10: 40
        "11111000", -- sample 11: 248
        "11110101", -- sample 12: 245
        "01111100", -- sample 13: 124
        "11001100", -- sample 14: 204
        "00100100", -- sample 15: 36
        "01101011", -- sample 16: 107
        "11101010", -- sample 17: 234
        "11001010", -- sample 18: 202
        "11110101", -- sample 19: 245
    );

    signal clk      : std_logic := '0';
    signal rst      : std_logic := '1';
    signal valid_in : std_logic := '0';
    signal x        : std_logic_vector(N-1 downto 0) := (others => '0');
    signal valid_out : std_logic;
    signal y        : std_logic_vector(18 downto 0);

begin

    -- Clock generation: 50 MHz (20 ns period)
    clk <= not clk after CLK_PER / 2;

    -- DUT instantiation
    dut: entity work.fir
        port map (
            clk      => clk,
            rst      => rst,
            valid_in  => valid_in,
            x         => x,
            valid_out => valid_out,
            y         => y
        );

end;
```

```

    );

-- Stimulus process
process
begin
    -- Hold reset for 2 cycles
    wait for 2 * CLK_PER;
    rst <= '0';

    -- Send 2 full square wave periods (2 x 32 = 64 samples)
    -- Each sample sent once every 8 clock cycles
    for period in 0 to 1 loop
        for i in 0 to 19 loop

            x          <= SQUARE_LUT(i);
            valid_in <= '1';
            wait for CLK_PER;          -- hold valid_in high for 1 cycle
            valid_in <= '0';
            wait for 7 * CLK_PER; -- wait remaining 7 cycles

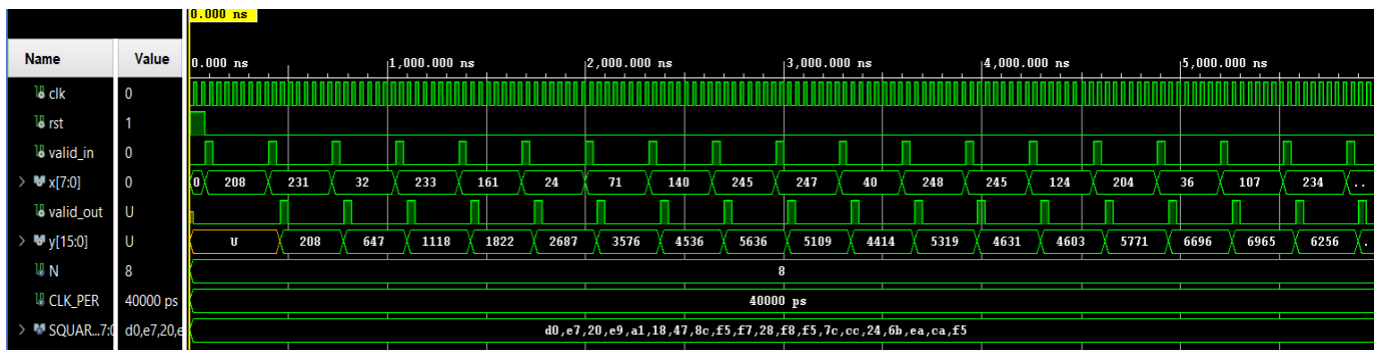
        end loop;
    end loop;

    -- Let last output settle
    wait for 10 * CLK_PER;

end process;
end Behavioral;

```

Τρέχοντας την προσομοίωση, λαμβάνουμε:

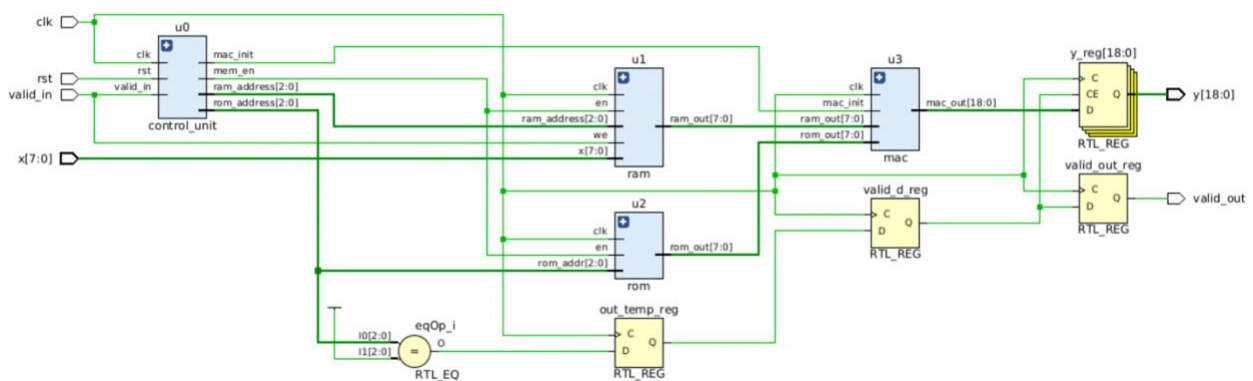


Το οποίο μπορεί να επιβεβαιωθεί και από τον παρακάτω πίνακα:

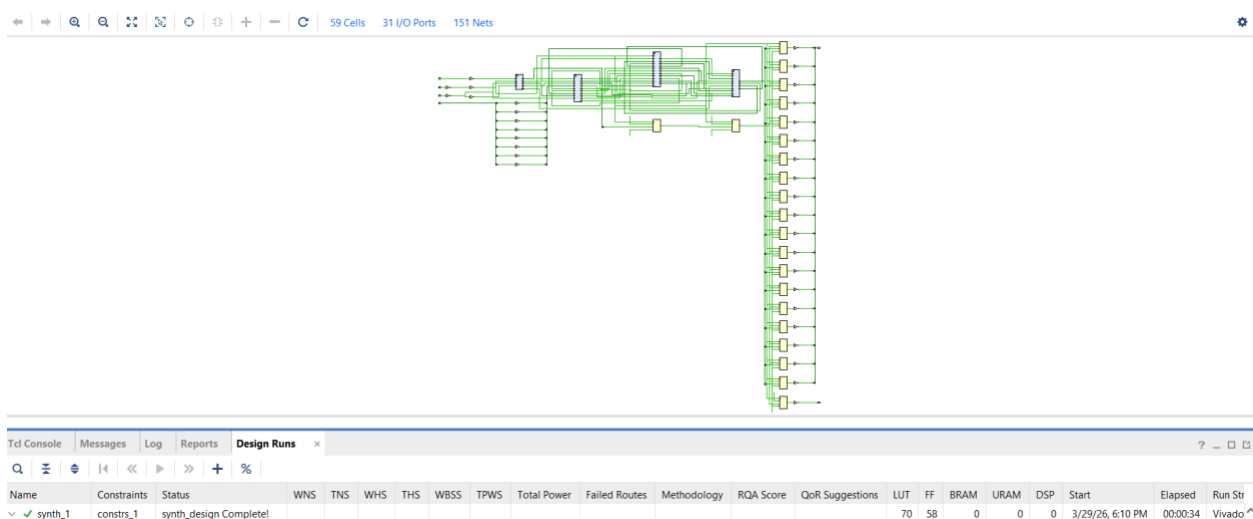
<i>h</i>	<i>x</i>	<i>y</i>
1	208	208
2	231	647
3	32	1118
4	233	1822
5	161	2687
6	24	3576

7	71	4536
8	140	5636
	245	5109
	247	4414
	40	5319
	248	4631
	245	4603
	124	5771
	204	6696
	36	6965
	107	6256
	234	5518

Με χρήση του Elaborated Design λαμβάνουμε:



Με τη χρήση του synthesis design λαμβάνουμε:



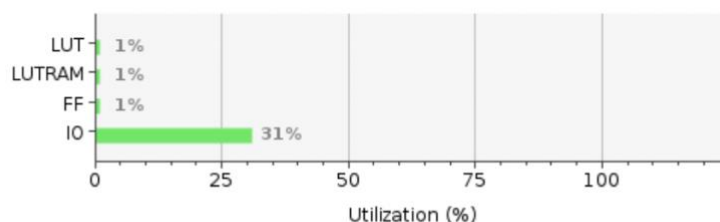
Επίσης, με τη βοήθεια του Report Timing Summary υπολογίζουμε το Critical Path :

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
↳ Path 1	∞	9	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[17]/D	5.547	2.483	3.064	∞
↳ Path 2	∞	9	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[18]/D	5.455	2.391	3.064	∞
↳ Path 3	∞	9	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[16]/D	5.434	2.370	3.064	∞
↳ Path 4	∞	8	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[13]/D	5.433	2.369	3.064	∞
↳ Path 5	∞	8	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[15]/D	5.414	2.350	3.064	∞
↳ Path 6	∞	8	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[14]/D	5.341	2.277	3.064	∞
↳ Path 7	∞	8	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[12]/D	5.320	2.256	3.064	∞
↳ Path 8	∞	7	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[9]/D	5.309	2.545	2.764	∞
↳ Path 9	∞	7	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[11]/D	5.290	2.526	2.764	∞
↳ Path 10	∞	7	4	20	u2/rom_out_reg[1]/C	u3/mac_temp_reg[10]/D	5.217	2.453	2.764	∞

Critical path = 5.547ns, άρα η μέγιστη συχνότητα λειτουργίας είναι **180.3 MHz**.

Τέλος, για να δούμε τους πόρους που χρησιμοποιήσαμε, θα τρέξουμε το Utilization του synthesis design:

Resource	Utilization	Available	Utilization %
LUT	70	17600	0.40
LUTRAM	8	6000	0.13
FF	58	35200	0.16
IO	31	100	31.00



Από το οποίο βλέπουμε ότι το fir φίλτρο μας χρειάστηκε 70 *look up tables (LUT)* και 58 *flip – flop (FF)*. Τα flip-flops χρησιμοποιήθηκαν για τους registers και οι LUT για την υλοποίηση λογικών πράξεων.